```
$ curl -X 'GET' http://localhost:8080/article/123
HTTP Method: 'GET'
Requested resource path is '/article/123'
Matched resource path and method to route 'read.article'
```

What we have created here is a rudimentary URL *router*. This is a very common concept, which you will encounter in many mainstream PHP frameworks, such as Symfony. The code in `index.php` simply examines the HTTP method and applies a regular expression to the contents of the `$_SERVER['REQUEST_URI']` superglobal. It's common convention for router libraries to consider both the request method and path when matching routes. In our case, the method and URI pattern are mapped to a route name (which we've implemented as a simple check on the keys of the `$routes` array). If the HTTP method and URI pattern match, then we could go on to call the function or object method intended to handle that particular request. In a production-quality framework, routing is usually config-driven (either in standalone config or, more commonly, by adding annotations or PHP attributes to classes and methods designed to interact with HTTP requests). I encourage you to investigate further, and the Symfony Routing component is an excellent place to start. https://symfony.com/doc/current/routing.html

Request Headers

An important aspect of HTTP requests to be aware of is the *headers*. These are implemented in the protocol simply as lines of text containing the header field name, a colon character, and the value of the header. They are, however, used for some fairly serious things, such as controlling server (or proxy) behavior, precisely describing the size and nature of request data, or the capabilities of the client… which makes them quite powerful little lines of text!

In terms of the protocol, headers come after the initial request line, and each `field: value` pair is separated by a new line. Header fields are standardized in the IANA HTTP Field Name Registry and are fully documented at https://www.iana.org/assignments/http-fields/http-fields.xhtml, which will make excellent bedtime reading if you are having trouble sleeping. Header fields can be grouped by their context; for example, the `Content-Length:` header field might describe the size (in bytes) of the response data or the payload data in the request. You might encounter the concept of "request headers" in terms of those fields only used for controlling a request (in other words, "request-only" headers such as `User-Agent:`, `Accept:`, or `Referrer:`), but for our purposes, here we're simply interested in all of the headers set by the client when the request is made.